Advanced Rust 2026 - Lab 5: Unsafe Rust Exercises

Lukáš Hozda

Spring 2026

Setup

Create a library project:

```
cargo new --lib unsafe_lab
```

Put the exercises into separate modules, for example:

```
pub mod raw_fixed_vec;
pub mod wire_header;
pub mod record_store;
```

Exercise 1: Manually Allocated Fixed Vector

Implement a fixed-capacity vector using manual allocation. This is not meant to replace `Vec`. The point is to work through the invariants that ordinary collections hide from you.

```
use std::alloc::{self, Layout};
use std::ptr::NonNull;

pub struct RawFixedVec<T> {
    ptr: NonNull<T>,
    len: usize,
    cap: usize,
}

impl<T> RawFixedVec<T> {
    pub fn with_capacity(capacity: usize) -> Self {
        todo!()
    }

    pub fn capacity(&self) -> usize {
        todo!()
    }

    pub fn len(&self) -> usize {
        todo!()
    }

    pub fn push(&mut self, value: T) -> Result<(), T> {
        todo!()
    }
```

```
    pub fn pop(&mut self) -> Option<T> {
        todo!()
    }

    pub fn get(&self, index: usize) -> Option<&T> {
        todo!()
    }

    pub fn get_mut(&mut self, index: usize) -> Option<&mut T> {
        todo!()
    }

    pub fn as_slice(&self) -> &[T] {
        todo!()
    }

    pub fn as_mut_slice(&mut self) -> &mut [T] {
        todo!()
    }
}

impl<T> Drop for RawFixedVec<T> {
    fn drop(&mut self) {
        todo!()
    }
}
```

For this exercise, handle zero capacity, but you may reject zero-sized types with an assertion. Supporting zero-sized types correctly is its own special case.

Required invariants:

1. `0 ≤ len ≤ cap`.
2. If `cap > 0`, `ptr` points to an allocation suitable for `cap` values of `T`.
3. Slots `0..len` are initialized values of type `T`.
4. Slots `len..cap` are uninitialized.
5. `push` writes exactly one previously uninitialized slot.
6. `pop` reads exactly one initialized slot and then treats it as uninitialized.
7. `Drop` drops only initialized elements and deallocates the allocation exactly once.

Allocation shape for nonzero capacity:

```
let layout = Layout::array::<T>(capacity).unwrap();
let raw = unsafe { alloc::alloc(layout) };
let ptr = NonNull::new(raw.cast::<T>())
    .unwrap_or_else(|| alloc::handle_alloc_error(layout));
```

Add tests for:

1. pushing until full and receiving the rejected value
2. popping in reverse order
3. `get`, `get_mut`, `as_slice`, and `as_mut_slice`
4. dropping a partially initialized vector
5. zero capacity

Exercise 2: Packed Header Parser

Implement a parser for a small binary packet header. The wire format is:

```
offset  size  field
0       2     magic bytes: ASCII "RS"
2       1     version, must be non-zero
3       1     flags
4       2     payload length, little-endian u16
6       4     checksum, little-endian u32
```

Only the lowest two flag bits are currently valid:

1. bit 0: compressed
2. bit 1: encrypted

All other flag bits must cause parsing to fail.

```rust
pub const HEADER_LEN: usize = 10;

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub struct Header {
    pub version: u8,
    pub compressed: bool,
    pub encrypted: bool,
    pub payload_len: u16,
    pub checksum: u32,
}

pub fn parse_header(bytes: &[u8]) -> Option<Header> {
    todo!()
}

pub fn split_packet(packet: &[u8]) -> Option<(Header, &[u8])> {
    todo!()
}
```

Use `std::ptr::read_unaligned` for the integer reads. Do not form references to possibly unaligned integer fields.

A reasonable helper shape is:

```rust
fn read_u16_le(bytes: &[u8], offset: usize) -> u16 {
    // TODO: use ptr::read_unaligned
    todo!()
}

fn read_u32_le(bytes: &[u8], offset: usize) -> u32 {
    // TODO: use ptr::read_unaligned
    todo!()
}
```

Requirements:

1. Reject inputs shorter than `HEADER_LEN`.
2. Reject packets whose magic bytes are not `RS`.
3. Reject version zero.
4. Reject flags with any bit outside `0b0000_0011` set.
5. Convert flags to booleans using comparisons or bit tests, not `transmute`.

6. `split_packet` must reject packets shorter than `HEADER_LEN + payload_len`.
7. `split_packet` returns only the payload slice, not trailing bytes after it.

Add tests for:

1. valid header with both flags set
2. invalid magic, version, and flags
3. too-short input
4. payload slicing with trailing bytes

Use this test data:

```
fn valid_packet() -> [u8; 15] {
    [
        b'R', b'S',
        3,
        0b0000_0011,
        5, 0,
        0x44, 0x33, 0x22, 0x11,
        b'h', b'e', b'l', b'l', b'o',
    ]
}
```

Useful assertions:

```
#[test]
fn parses_valid_header() {
    let packet = valid_packet();
    let header = parse_header(&packet).unwrap();
    assert_eq!(header.version, 3);
    assert!(header.compressed);
    assert!(header.encrypted);
    assert_eq!(header.payload_len, 5);
    assert_eq!(header.checksum, 0x1122_3344);
}

#[test]
fn rejects_invalid_fields() {
    let mut packet = valid_packet();
    packet[0] = b'X';
    assert_eq!(parse_header(&packet), None);

    let mut packet = valid_packet();
    packet[2] = 0;
    assert_eq!(parse_header(&packet), None);

    let mut packet = valid_packet();
    packet[3] = 0b0000_0100;
    assert_eq!(parse_header(&packet), None);
}

#[test]
fn splits_payload_without_trailing_bytes() {
    let mut packet = valid_packet().to_vec();
    packet.extend_from_slice(b"ignored");

    let (header, payload) = split_packet(&packet).unwrap();
```

```
    assert_eq!(header.payload_len, 5);
    assert_eq!(payload, b"hello");
}
```

Exercise 3: Run Miri on Broken Code

Create `src/record_store.rs` and put this module into it. The code is meant to look like ordinary low-level parsing and storage code. It compiles, and ordinary tests pass on a typical development machine, but it is not sound.

```
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
pub struct Entry {
    pub tag: u8,
    pub enabled: bool,
}

pub fn decode_entry(bytes: [u8; 2]) -> Entry {
    Entry {
        tag: bytes[0],
        enabled: unsafe { std::mem::transmute::<u8, bool>(bytes[1]) },
    }
}

#[repr(C)]
#[derive(Clone, Copy)]
pub struct DiskRecord {
    pub tag: u8,
    pub counter: u32,
}

pub fn record_bytes(record: DiskRecord) -> [u8; std::mem::size_of::<DiskRecord>()] {
    unsafe { std::mem::transmute(record) }
}

pub fn increment_twice() -> i32 {
    let mut value = 40_i32;
    let raw = &mut value as *mut i32;
    let reference = &mut value;
    *reference += 1;
    unsafe {
        *raw += 1;
    }
    value
}

#[repr(C, packed)]
pub struct PackedHeader {
    pub tag: u8,
    pub len: u16,
}

pub fn packed_len() -> u16 {
    let header = PackedHeader { tag: 1, len: 0x1234_u16.to_le() };
    let ptr = std::ptr::addr_of!(header.len);
```

```
    unsafe { u16::from_le(ptr.read()) }
}
```

Add this to `src/lib.rs`:

```
pub mod record_store;
```

Add tests:

```
#[cfg(test)]
mod tests {
    use super::record_store::*;

    #[test]
    fn decodes_entry() {
        let entry = decode_entry([7, 3]);
        assert_eq!(entry.tag, 7);
        assert!(entry.enabled);
    }

    #[test]
    fn serializes_record() {
        let bytes = record_bytes(DiskRecord {
            tag: 9,
            counter: 0x1122_3344,
        });
        assert_eq!(bytes[0], 9);
        assert_eq!(&bytes[4..8], &0x1122_3344_u32.to_ne_bytes());
    }

    #[test]
    fn updates_counter() {
        assert_eq!(increment_twice(), 42);
    }

    #[test]
    fn reads_packed_len() {
        assert_eq!(packed_len(), 0x1234);
    }
}
```

Run the tests normally first:

```
cargo test
```

Then run the failing cases under Miri, one at a time:

```
rustup +nightly component add miri
cargo +nightly miri test decodes_entry
cargo +nightly miri test serializes_record
cargo +nightly miri test updates_counter
cargo +nightly miri test reads_packed_len
```

For each failure, identify where Miri reports the first violation and which operation first makes the program undefined. Then write the smallest sound replacement for that operation.